Motores de Juegos

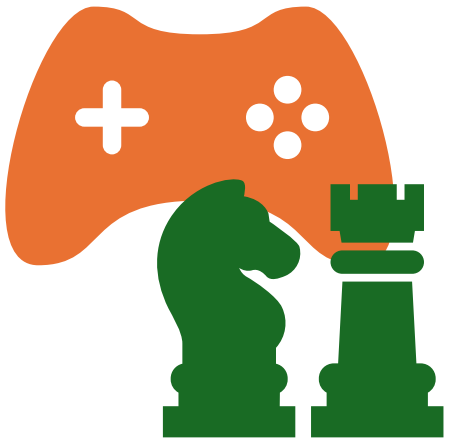
Unidad de Trabajo 4

Contenidos

1. Arquitectura de videojuegos 2D y 3D
2. Motores de juegos: concepto y componentes
3. Entornos de desarrollo de juegos
4. Comparativa de motores (2D y 3D)
5. Análisis estructural de un videojuego existente
6. Escenas gráficas: representación lógica y espacial

Introducción y arquitectura de videojuegos

- Un videojuego puede entenderse como un **sistema interactivo en tiempo real** que combina:
  - Entrada del usuario
  - Procesamiento lógico
  - Representación gráfica
  - Salida audiovisual



- Desde el punto de vista de la programación, un juego no es una sucesión lineal de instrucciones, sino un **ciclo continuo de ejecución**, donde el estado del juego se actualiza constantemente mientras está en funcionamiento.

Arquitectura de un videojuego

- **Bucle principal** (game loop)
  - Es el núcleo del funcionamiento del videojuego. Se trata de un ciclo que se ejecuta repetidamente mientras el juego está activo:
    - Captura de entrada (Input): Teclado / Pantalla táctil / Ratón / Sensores (acelerómetro, giroscopio, etc.)
    - Actualización de la lógica del juego:
      - Movimiento de personajes
      - Colisiones
      - Reglas del juego
      - Inteligencia artificial
    - Renderizado (dibujado en pantalla): Gráficos / Animaciones / Efectos visuales
  - Este ciclo se repite decenas de veces por segundo (FPS), lo que permite la sensación de fluidez y tiempo real.

Arquitectura de un videojuego

- **Gestión de estados** (menús, juego, pausa).
  - Los juegos suelen organizarse en estados, cada uno con un comportamiento específico, por ejemplo:
    - Menú principal
    - Opciones
    - Pantalla de carga
    - Juego en curso
    - Pausa
    - Fin de partida
  - Tienen su propia lógica y controlan qué se dibuja en pantalla o qué entradas del usuario son válidas.

Arquitectura de un videojuego

- **Sistema de entrada** (*Input System*)
  - El sistema de entrada se encarga de:
    - Detectar acciones del usuario
    - Traducirlas a eventos del juego
  - Ejemplos:
    - Pulsar un botón → saltar
    - Deslizar el dedo → mover un personaje
    - Inclinar el dispositivo → girar la cámara
  - En juegos móviles, este sistema cobra especial importancia debido al uso de pantallas táctiles y sensores.

Arquitectura de un videojuego

- **Lógica del juego**
  - La lógica del juego define las reglas:
    - Qué puede y qué no puede hacer el jugador
    - Cómo reaccionan los elementos del juego
    - Cuándo se gana o se pierde
  - Incluye:
    - Movimiento de entidades
    - Sistema de colisiones
    - Puntuación
    - Vidas
    - Eventos y condiciones
  - Esta parte es independiente de la representación gráfica, lo que permite separar claramente qué ocurre de cómo se muestra.

Arquitectura de un videojuego

- **Renderizado gráfico**
  - El renderizado es el proceso de dibujar el estado actual del juego en pantalla.
  - Incluye:
    - Sprites o modelos 3D
    - Animaciones
    - Efectos visuales
    - Cámaras y perspectiva
  - El motor gráfico se encarga de convertir los datos lógicos del juego en imágenes visibles para el jugador.

Arquitectura de un videojuego

- **Juegos 2D vs 3D**

Aspecto	Juegos 2D	Juegos 3D
Coordenadas	X, Y	X, Y, Z
Gráficos	Sprites	Modelos 3D
Cámara	Simple o fija	Dinámica y compleja
Complejidad	Baja–media	Media–alta
Rendimiento	Más eficiente	Más exigente

Arquitectura de un videojuego

- Esta arquitectura no depende del motor
- Los motores de juegos:
  - Implementan esta arquitectura internamente
  - Automatizan tareas repetitivas
  - Permiten al desarrollador centrarse en la lógica del juego
- Comprender la arquitectura es clave para:
  - Elegir el motor adecuado
  - Entender cómo funciona internamente
  - Programar de forma más eficiente

Motores de videojuegos

Un **motor de juegos** es un **conjunto de herramientas software** que proporciona una base común para el desarrollo de videojuegos, evitando que el programador tenga que implementar desde cero los elementos básicos de la arquitectura de un juego.

- El motor:
  - Implementa internamente el game loop
  - Gestiona gráficos, sonido, físicas y entrada
  - Ofrece un entorno de trabajo integrado (editor + librerías)
  - Su objetivo principal es acelerar el desarrollo y reducir la complejidad técnica, permitiendo centrarse en la lógica y el diseño del juego.

Motores de videojuegos

- Los motores de juegos **materializan** la arquitectura general.

- **Ventajas** de usar un motor de juegos
  - Reducción del tiempo de desarrollo
  - Mayor calidad técnica
  - Portabilidad entre plataformas
  - Uso de herramientas profesionales
  - Escalabilidad del proyecto

Motores de videojuegos

- **Sistema de eventos**
  - Gestiona las interacciones del jugador con el juego.
  - Tipos de entrada:
    - Teclado y ratón
    - Pantalla táctil (toques, gestos)
    - Sensores del dispositivo
    - Mandos externos
  - El motor traduce estas entradas en:
    - Eventos
    - Acciones configurables
  - Esto permite adaptar fácilmente el juego a distintas plataformas.

Motores de videojuegos

- **Sistema de componentes**
  - La mayoría de motores modernos lo utilizan, con los conceptos clave:
    - Objeto del juego: entidad genérica (jugador, enemigo, obstáculo)
    - Componentes: funcionalidades que se añaden al objeto
  - Ejemplos de componentes asociados a un objeto:
    - Transformación (posición, rotación)
    - Renderizado
    - Colisión
    - Script de comportamiento
    - Audio
  - Ventaja: gran flexibilidad y reutilización.

Motores de videojuegos

- **Sistema de scripting**
  - Es la parte donde el desarrollador programa el comportamiento del juego.
  - Características:
    - Uso de lenguajes de alto nivel (C#, GDScript, C++, Java)
    - Acceso a eventos del motor
    - Interacción con objetos y componentes
  - Permite implementar:
    - Reglas del juego
    - IA básica
    - Control del jugador
    - Eventos y condiciones

Motores de videojuegos

- **Motor físico y sistema de colisiones**
  - Este componente simula el comportamiento físico del juego.
  - Incluye:
    - Detección de colisiones
    - Gravedad y otras fuerzas
    - Cuerpos rígidos, blandos y partículas
    - Velocidad y aceleración
  - En móviles:
    - Suele usarse una física simplificada
    - Se prioriza el rendimiento
  - Permite:
    - Evitar cálculos manuales complejos
    - Comportamientos más realistas o coherentes

Motores de física populares:
•**Bullet Physics:** Código abierto, usado en simulaciones y videojuegos.
•**Havok:** Usado en juegos AAA como Halo y Assassin's Creed.
•**NVIDIA PhysX:** Especializado en simulaciones complejas

Motores de videojuegos

- **Motor gráfico**
  - Es el encargado de la representación visual del juego (renderizado)
  - Tiene por función:
    - Dibujar sprites (2D) o modelos (3D)
    - Gestionar cámaras
    - Aplicar transformaciones (posición, rotación, escala)
    - Iluminación, sombras y efectos

Un **sprite** era una imagen (mapa de bits) gestionada por un HW especializado, independiente de la CPU, usado para los personajes de videojuegos ya que permitía gestionarlos de manera independiente a los fondos. Actualmente, suele hacer referencia a cualquier imagen de mapa de bits presente en pantalla, generalmente refiriéndose a los personajes del juego.

Motores de videojuegos

- **Motor gráfico (RENDERIZADO)**
  - El renderizado en videojuegos es un proceso crucial que permite crear imágenes y gráficos en tiempo real.
  - APIs populares: OpenGL, DirectX, Vulkan, Metal.
  - Etapas del Pipeline
    - Transformación de Modelos: Convierte los modelos desde sus coordenadas locales a coordenadas del mundo.
    - Transformación de Vista (Cámara): Ajusta la perspectiva para simular lo que ve el jugador.
    - Proyección: Convierte coordenadas 2D/3D.
    - Rasterización: Divide primitivas (triángulos) en píxeles que se pintan en pantalla.
    - Fragment Shading: Aplica efectos visuales como texturas, luces y sombras.

Motores de videojuegos

- **Sistema de audio**
  - Forma parte de la representación audiovisual junto al motor gráfico.
  - Encargado de la reproducción de:
    - Música
    - Efectos de sonido
    - Audio ambiental
  - Funciones habituales:
    - Reproducción en bucle
    - Control de volumen
    - Audio espacial (especialmente en 3D)
  - El audio es clave para la experiencia de usuario y la inmersión.

Motores de videojuegos

- **Sistema de escenas**
  - Las escenas representan distintas partes del juego:
    - Menú principal
    - Nivel 1, Nivel 2…
    - Pantalla de fin de partida
  - Cada escena contiene:
    - Objetos del juego
    - Componentes asociados
    - Lógica específica
  - Este sistema facilita:
    - Organización del proyecto
    - Reutilización de elementos
    - Gestión de estados del juego

Motores de videojuegos

- **Gestión de Memoria**
  - Minimiza el uso excesivo de memoria cargando recursos bajo demanda.
  - Implementa algoritmos de caché y técnicas de streaming.
- **Procesamiento Paralelo**
  - Divide tareas como físicas y renderizado en hilos separados.
- **Uso de Shaders**
  - Los shaders son programas ejecutados en la GPU para efectos visuales avanzados.
  - Tipos comunes:
    - Vertex Shader: Modifica la geometría.
    - Fragment Shader: Procesa píxeles.

Entornos de desarrollo y motores de videojuegos

- **Entornos de desarrollo de videojuegos:**
  - Editores visuales
  - IDEs integrados
- **Motores de juegos habituales:**
  - Unity (Pokémon GO)

  - Godot (Brotato)

  - Unreal Engine (Gears of War)

  - LibGDX (Pathway)

  - Phaser (Pathway)

Entornos de desarrollo y motores de videojuegos

- **Comparativa de motores**

Criterio	Unity	Godot	Unreal	LibGDX	Phaser
Tipo	Motor completo	Motor completo	Motor completo	Framework	Framework
Enfoque principal	2D / 3D	2D / 3D	3D avanzado	2D / 3D ligero	2D Web
Lenguaje	C#	GDScript / C#	C++ / Blueprints	Java	JavaScript/TypeScript
IDE	Visual Studio / Rider	Editor scripts integrado	Visual Studio	IntelliJ / Eclipse	VS Code
Editor visual	Scene/Game View	Editor escenas/nodos	Level Editor	✕	✕
Gestión de escenas	✓	✓	✓	Manual	Manual
Motor físico integrado	✓	✓	✓	Librerías externas	Librerías
Multiplataforma	Muy alta	Alta	Alta	Alta	Web
Rendimiento en móviles	Alto	Muy alto	Medio	Muy alto	Alto
Curva de aprendizaje	Media	Baja	Alta	Media-Alta	Media
Comunidad	Muy grande	Media	Grande	Media	Grande
Licencia	Comercial	Libre	Comercial	Libre	Libre

Entornos de desarrollo y motores de videojuegos

- **Retos en la Arquitectura de Motores de Juegos**
  - Multiplataforma:
    - Diseñar para PC, consolas y móviles requiere modularidad.
  - Balance entre Realismo y Rendimiento:
    - Escoger simulaciones simplificadas para evitar caídas de FPS.
  - Evolución Tecnológica:
    - Soporte para nuevas tecnologías como Ray Tracing y VR.